DALHOUSIE UNIVERSITY

Support Vector Machines

Margins, kernels, and tuning inside cross-validation

Prof. Gabriel Spadon, PhD

Faculty of Computer Science

spadon@dal.ca

1 Recap: how LDA draws its line

One score, two means LDA, as previously seen, turns each case into a single score, a weighted sum $z = w_1X_1 + w_2X_2 + \cdots$, and chooses the weights by Fisher's separation score, which pushes the two class means apart while keeping each class tight. With equal class sizes and one shared spread, the boundary sits where a case is equally close to both means. Figure 1 draws it on the twelve cases used throughout this handout. The class means are $(1.5, 1.5)$ and $(5, 5)$, and LDA's boundary is $X_2 = 6.5 - X_1$.

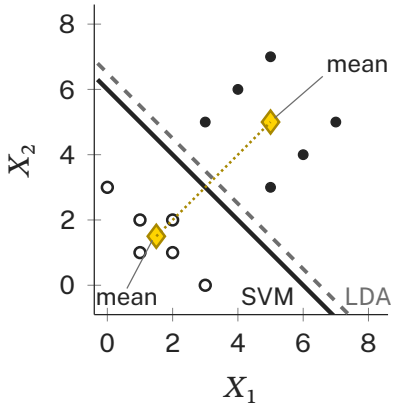


Figure 1. LDA's boundary (dashed), halfway between the two class means (diamonds), and the SVM's boundary (solid) on the same twelve cases.

The same data, a different question LDA asks where the classes sit on average and how spread they are. Its line comes from the means and the shared spread, so every case helps place it, and it assumes each class is a roughly Normal cloud. The SVM asks which line leaves the most empty room between the classes. Its line comes only from the cases nearest the gap, and it assumes nothing about the shape of each class. On these twelve cases both lines run in the same direction, and LDA's sits half a unit higher. Move a case far from the gap and LDA's line moves, because a mean moves, while the SVM's does not. Move a case near the gap and the SVM's line moves at once, while LDA's barely does.

2 Where support vector machines sit

Earlier classifiers sit at opposite ends of a dial. kNN draws a boundary from local votes and can bend around anything. LDA draws a straight boundary from the class means and barely moves when a single case moves. A support vector machine (SVM) sits between them. It starts from a straight boundary, chooses the one that leaves the widest gap between the classes, and bends it only as far as two settings allow.

Table 1 lists the ideas of the class, the problem each answers, and what you set. The cost C and the kernel are model choices. Scaling and the tuning protocol are workflow choices, and they come straight from earlier work.

Table 1. The ideas of the class, the problem each answers, and what you set

Idea	The problem it answers	What you set or check
Widest margin	many boundaries fit the training data	nothing; the fit chooses it
Soft margin	real classes overlap	the cost C
Kernel	the boundary is not straight	the kernel and its γ
Scaling	distances depend on units	a scaler fitted on training rows
Tuning	C and γ interact	a grid chosen inside, scored outside
Class weights	a rare class is ignored	weights and a class-aware metric

3 The widest street

What it is for When two classes can be separated by a straight line, many lines do it, and every one of them has zero training error. Training error cannot choose between them. The SVM chooses the line in the middle of the **widest street** that separates the classes [4]. On the running dataset three such lines split the twelve cases, with streets 0.89, 1.79 and 2.83 units wide, twice the distance from each line to its nearest case (Figure 2). The widest, $x_1 + x_2 = 6$, is the safest bet for new cases. A new case resembles the training cases without sitting exactly on them. With the widest street it can drift up to half the width, 1.41 units, towards the other class and still fall on its own side, while with the narrowest street a drift of 0.45 units is enough to misclassify it.

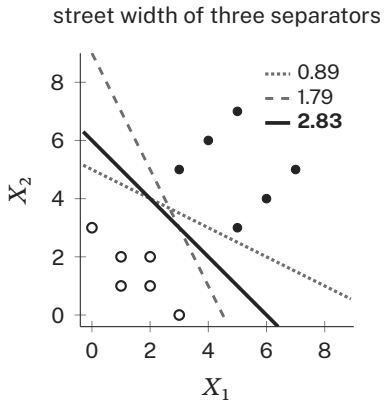


Figure 2. Three lines separate the twelve running cases with zero training error. Their streets are 0.89, 1.79 and 2.83 units wide, and the SVM takes the widest.

The running dataset Twelve cases in two dimensions, six per class (Table 2). The fitted line is $x_1 + x_2 = 6$, the edges of the street are $x_1 + x_2 = 8$ and $x_1 + x_2 = 4$, and the street is 2.83 units wide (Figure 3). That width is the **margin**.

Table 2. Coordinates of the twelve cases in the running dataset

$y = +1$	$(5, 3)$, $(3, 5)$, $(6, 4)$, $(4, 6)$, $(7, 5)$, $(5, 7)$
$y = -1$	$(2, 2)$, $(1, 1)$, $(2, 1)$, $(1, 2)$, $(3, 0)$, $(0, 3)$

Support vectors Three of the twelve cases lie on the edges of the street, $(5, 3)$ and $(3, 5)$ on one edge and $(2, 2)$ on the other (Figure 3). These are the **support vectors**. Delete the other nine cases and refit, and the same line comes back. The fitted model stores only these cases, which is why the method carries their name.

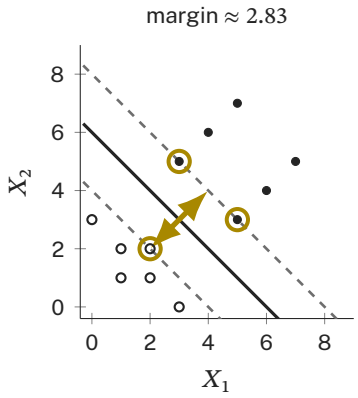


Figure 3. The street on the running dataset, 2.83 units wide. Filled points are one class and open points the other. The solid line is the boundary, the dashed lines are the edges of the street, and the three ringed cases on the edges are the support vectors.

Two consequences matter in practice. An SVM is *robust to cases far from the boundary and sensitive to cases on it*. Moving $(5, 7)$ while keeping it outside the street changes nothing, but moving $(2, 2)$ moves the boundary with it. And the street is measured in the units of the predictors, which Section 6 returns to.

How a new case is classified The fitted line is a weighted sum of the predictors plus a constant, the same shape as LDA's score,

$$f(\mathbf{x}) = w_1 x_1 + w_2 x_2 + b = 0.5\, x_1 + 0.5\, x_2 - 3.$$

On the line $f = 0$, on the two edges of the street $f = +1$ and $f = -1$, and the sign of f gives the predicted class. The software calls f the *decision value*. Three new cases show how to read it (Figure 4). For $(4, 4)$, $f = 2+2-3 = 1.0$, so the case sits on the edge of the filled side. For $(3, 2)$, $f = 1.5+1-3 = -0.5$, so it is open but inside the street, closer to the boundary than any training case. For $(6, 6)$, $f = 3.0$, deep in the filled side. The size of f ranks cases by how far they sit from the boundary, so an ROC curve can be drawn from it, but it is not a probability. The width of the street follows from the same weights, $2/\|\mathbf{w}\| = 2/\sqrt{0.5^2 + 0.5^2} = 2.83$.

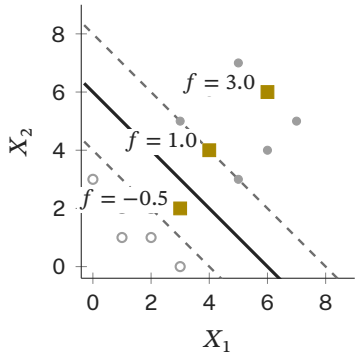


Figure 4. The decision value f of three new cases, read against the line ($f = 0$) and the two edges of the street ($f = \pm 1$).

R

```r
new <- data.frame(x1 = c(4, 3, 6), x2 = c(4, 2, 6))
f <- predict(s, new, decision.values = TRUE)
round(attr(f, "decision.values"), 2)
## 1.0 -0.5 3.0, so classes 1, -1, 1
```

Python

```python
new = np.array([[4, 4], [3, 2], [6, 6]])
print(s.decision_function(new).round(2))
# [ 1.  -0.5  3. ], so classes 1, -1, 1
```

Listing 1. R and Python: decision values of three new cases

Code Both calls fit the running data with a linear kernel and a very large cost, which allows no case inside the street. Each prints the support vectors, the line, and the width of the street, computed from the fitted weights.

R

```r
library(e1071)
d <- data.frame(
  x1 = c(5, 3, 6, 4, 7, 5, 2, 1, 2, 1, 3, 0),
  x2 = c(3, 5, 4, 6, 5, 7, 2, 1, 1, 2, 0, 3),
  y  = factor(rep(c(1, -1), each = 6),
              levels = c(1, -1)))
s <- svm(y ~ x1 + x2, d, kernel = "linear",
         cost = 1e5, scale = FALSE)
s$index        # 1 2 7, the support vectors
w <- t(s$coefs) %*% s$SV       # (0.5, 0.5)
-s$rho         # b = -3
2 / sqrt(sum(w^2))             # margin 2.83
```

Python

```python
import numpy as np
from sklearn.svm import SVC

X = np.array([[5, 3], [3, 5], [6, 4], [4, 6],
              [7, 5], [5, 7], [2, 2], [1, 1],
              [2, 1], [1, 2], [3, 0], [0, 3]])
y = np.array([1] * 6 + [-1] * 6)
s = SVC(kernel="linear", C=1e5).fit(X, y)
print(s.support_)     # [6 0 1], zero-based row numbers
print(s.coef_, s.intercept_)        # (0.5, 0.5), -3
print(2 / np.linalg.norm(s.coef_))  # margin 2.83
```

Listing 2. R and Python: the widest street on the running data

Differences to know

- R numbers rows from 1 and Python from 0, so `s$index` is `1 2 7` and `support_` is `[6 0 1]`.
- `svm()` standardizes every numeric column by default. `scale = FALSE` is needed here so that the line is in the original units. `SVC` never standardizes.

4 Soft margins and the cost C

What it is for Real classes overlap, and one badly placed case can ruin a street that insists on separating everything. The soft margin lets some cases step into the street, or across the line, in exchange for a wider and more stable street. The cost C sets the price.

What one outlier does Figure 5 uses 61 simulated cases of income against lot size, both standardized, after the riding-mower example of Boehmke and Greenwell [1, chapter 14]. On the 60 well-separated cases the street is 1.86 wide. One added case on the wrong side forces a street that still separates everything to shrink to 0.44 and swings the boundary. LDA fitted to the same cases barely moves, because its boundary comes from the class means.

The rule Each case may intrude into the street, or cross the boundary, if it pays a **fine** that grows with how far it intrudes, measured from the edge of the street on its own side (Table 3 and Figure 7). In the units of f the intrusion of a case of class $y = \pm 1$ is $\max(0,\ 1 - y\,f)$, which is 0 for a case outside the street, between 0 and 1 for a case inside the street on its own side, and above 1 for a misclassified case. The cost C sets the fine per unit of intrusion, and the fit balances two wishes, a wide street against C times the total intrusion. Every case on or inside the street is now a support vector, including the misclassified ones.

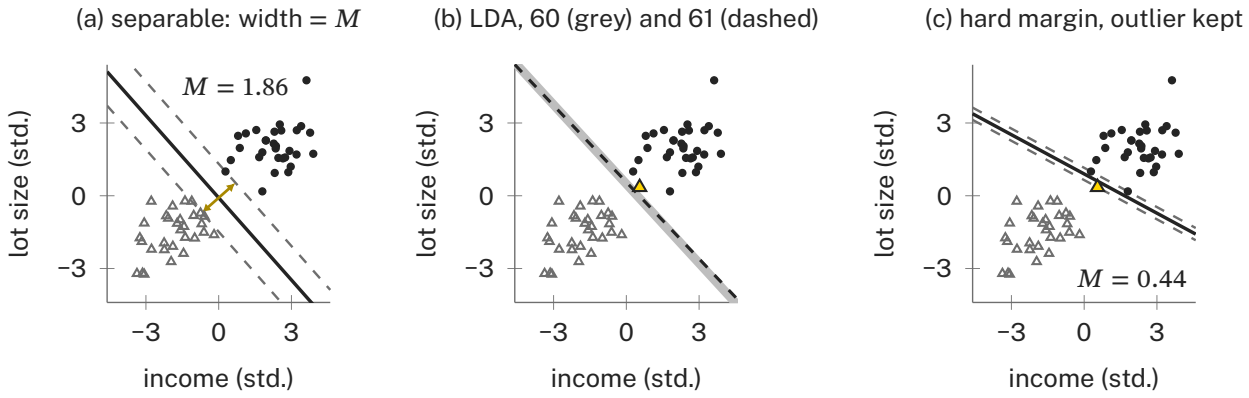


Figure 5. (a) The widest street on 60 separable cases. (b) LDA fitted to the 60 cases (grey) and to all 61 (dashed). (c) A street that must separate all 61 shrinks from 1.86 to 0.44. After Boehmke and Greenwell [1, chapter 14].

On the running cases Add one filled case at $(2, 3)$, among the open ones, so that no straight line separates the thirteen cases (Figure 6). With $C = 1$ the fitted line is $0.4\,x_1 + 0.8\,x_2 = 3.4$, the street is 2.24 wide, and $(2, 3)$ gets $f = 0.4 \times 2 + 0.8 \times 3 - 3.4 = -0.2$. It is misclassified and pays an intrusion of $1 - (-0.2) = 1.2$. Every other case stays outside the street, so the total intrusion is 1.2.

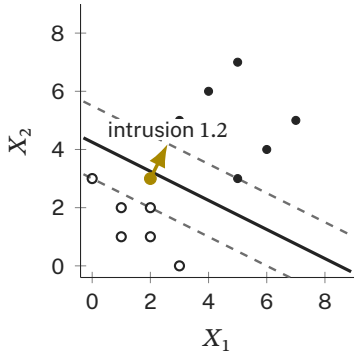


Figure 6. The thirteen cases with the soft margin at $C = 1$. The filled case $(2, 3)$ crosses the line, and the arrow is its intrusion past its own edge.

Table 3. Reading the intrusion of one case

Where the case sits	What it pays
outside the street, on its own side	nothing
inside the street, still on its own side	a small fine, and it is still correct
exactly on the boundary	a fine of one half-street
across the boundary	more than one half-street, and it is misclassified

C as a dial Table 4 uses the same 61 cases and the same linear kernel, changing only C, and Figure 8 plots the width of the street against C.

- **Small C.** Fines are cheap. The street is wide, many cases become support vectors, and the boundary is set by the bulk of each class. Too small, and it underfits.
- **Large C.** Fines are steep. The street narrows, few cases decide the boundary, and one outlier can swing it. Too large, and it overfits.

Figure 9 shows both ends on the 61 cases. C is a setting, like K in kNN. It is chosen by cross-validation, never by training error, because training error only falls as C grows.

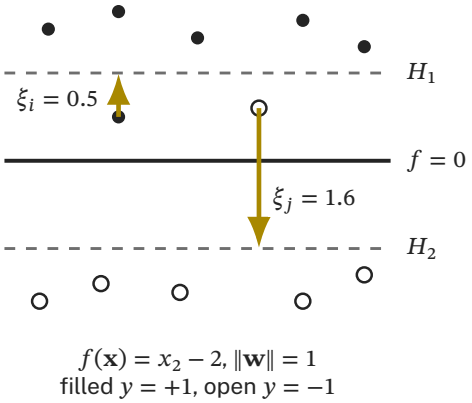


Figure 7. Each arrow is one case's intrusion past the edge of the street on its own side. A case outside the street pays nothing.

Table 4. Effect of cost C on the street width and the support vector count

cost C	margin width	support vectors	training errors
0.01	4.90	26	1
0.10	3.06	8	1
1	2.08	4	1
10	0.45	3	0
100	0.44	3	0

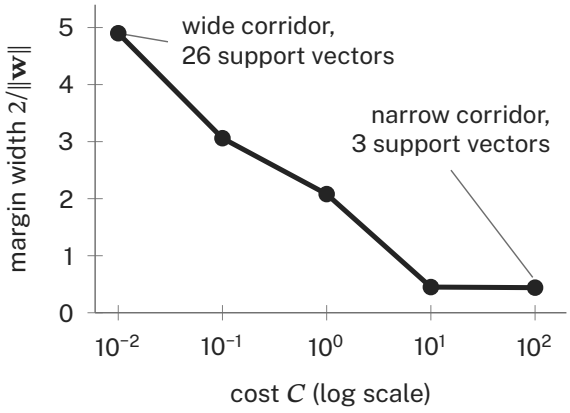


Figure 8. Street width against cost C on a log scale, for the 61 cases of Table 4.

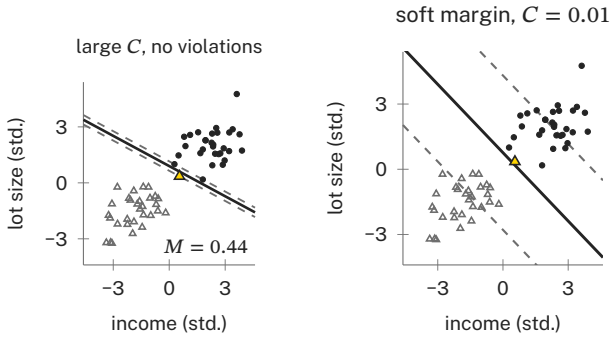


Figure 9. Left, a street that may not be violated bends around the outlier, with 3 support vectors. Right, $C = 0.01$ gives a wide street that ignores the outlier, with 26 support vectors and 1 training error.

Code Both calls add the case $(2, 3)$, which belongs with the filled class but sits among the open one, to the twelve running cases and sweep C.

R

```r
d2 <- rbind(d, data.frame(x1 = 2, x2 = 3,
            y = factor(1, levels = c(1, -1))))
for (C in c(0.01, 0.1, 1, 10, 100)) {
  s <- svm(y ~ x1 + x2, d2, kernel = "linear",
           cost = C, scale = FALSE)
  w <- drop(t(s$coefs) %*% s$SV)
  cat(C, round(2 / sqrt(sum(w^2)), 2),  # margin
      sum(fitted(s) != d2$y), "\n")  # errors
}
## 0.01 8.08 1
## 0.1 3.54 1
## 1 2.24 1
## 10 0.89 0
## 100 0.89 0
```

Python

```python
X2 = np.vstack([X, [2, 3]])
y2 = np.append(y, 1)
for C in [0.01, 0.1, 1, 10, 100]:
    s = SVC(kernel="linear", C=C).fit(X2, y2)
    print(C, round(2 / np.linalg.norm(s.coef_), 2),  # margin
          (s.predict(X2) != y2).sum())  # errors
```

Listing 3. R and Python: sweeping the cost with one outlier added

Both print street widths 8.08, 3.54, 2.24, 0.89, 0.89 and training errors $1, 1, 1, 0, 0$. The total intrusion falls with them, 5.88, 1.6, 1.2, 0 and 0. At $C = 0.01$ fines are so cheap that a wide street leaves all thirteen cases inside it. From $C = 10$ the fine on $(2, 3)$ costs more than a narrow street, and the fit that separates everything returns, with a street under a third of the original 2.83.

5 Kernels: curved boundaries from similarity

What it is for Most boundaries are not straight, even with a soft margin. A kernel lets the SVM draw a curved boundary without changing the fitting algorithm.

A derived feature can straighten a boundary In Figure 10 one class forms a disc and the other a surrounding ring, and no straight line separates them. Add one derived feature, $x_1^2 + x_2^2$, the squared distance from the centre. The inner class has small values and the outer class large ones, so a flat cut separates them, and read back on the original axes the cut is a circle of radius 0.69. This is ordinary feature engineering.

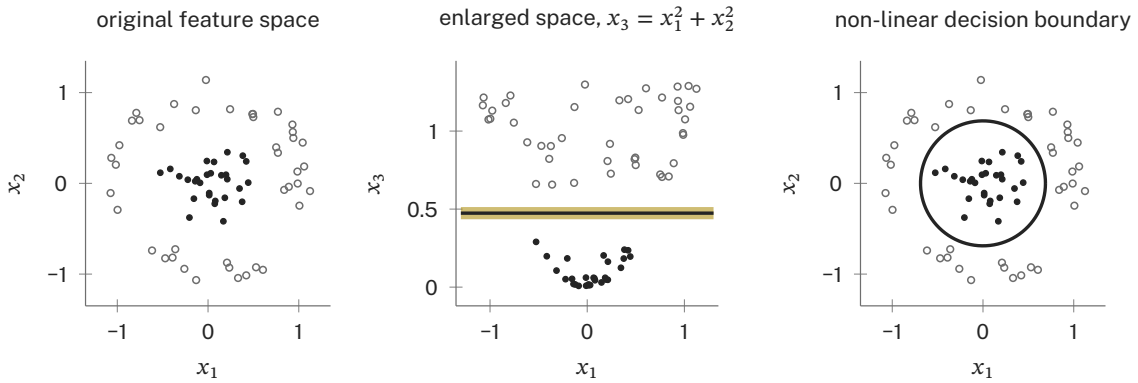


Figure 10. Left, no straight line separates the classes. Middle, the derived feature $x_1^2 + x_2^2$ separates them with a horizontal line. Right, the line mapped back is a circle of radius 0.69. After Boehmke and Greenwell [1, chapter 14].

The kernel does it for you Building derived features by hand is slow, and you would have to guess which ones help. The fitted SVM never needs the derived features themselves. It needs only one

number for each pair of cases, how similar they are, and a **kernel** computes that number directly from the original columns [2]. Choosing a kernel is choosing a notion of similarity. The fitted model is then a weighted vote of its support vectors,

$$f(\mathbf{x}) = \sum_{\text{support vectors}} \text{weight} \times \text{similarity to } \mathbf{x} \; + \; \text{constant},$$

where the fit learns one weight per support vector, positive for the filled class and negative for the open one. This is a smoothed relative of kNN. kNN lets the K nearest training cases vote equally, while an SVM lets only the support vectors vote, each by its learned weight scaled by its similarity.

The kernels in the libraries Two are in common use.

- **Linear**, the similarity of the original columns as they are. A straight boundary. It suits data with very many columns, as with text, where a straight boundary is already flexible enough.
- **Radial** (`"rbf"` in scikit-learn), whose kernel parameter γ sets how fast similarity falls, with

  $$\text{similarity} = e^{-\gamma \times (\text{squared distance between the cases})}.$$

  It is 1 for identical cases and falls towards 0 with distance. It is the default in both libraries and the usual first choice.

The vote for one case Fit a radial SVM with $\gamma = 0.05$ to the twelve running cases. It keeps five support vectors. Table 5 classifies the new case $(4, 4)$. Each squared distance becomes a similarity, the similarity is multiplied by the support vector's weight, and the products are added. The two filled support vectors at squared distance 2 contribute $+1.46$ each. The open $(2, 2)$ has the largest weight but sits farther, at squared distance 8, and contributes -1.88. The two far open cases add almost nothing. The sum is 0.86, the constant is 0.18, and $f(4, 4) = 1.04$, so the case is filled.

Table 5. The radial SVM's vote for the new case $(4, 4)$, $\gamma = 0.05$

Support vector	Squared distance	Similarity	Weight	Product
$(5, 3)$, filled	2	0.905	$+1.61$	$+1.46$
$(3, 5)$, filled	2	0.905	$+1.61$	$+1.46$
$(2, 2)$, open	8	0.670	-2.81	-1.88
$(3, 0)$, open	17	0.427	-0.21	-0.09
$(0, 3)$, open	17	0.427	-0.21	-0.09

R

```r
r <- svm(y ~ x1 + x2, d, kernel = "radial",
         gamma = 0.05, cost = 1e5, scale = FALSE)
r$index                  # 1 2 7 11 12
sim <- exp(-0.05 * colSums((t(r$SV) - c(4, 4))^2))
round(sim, 3)            # .905 .905 .670 .427 .427
round(r$coefs * sim, 2)  # 1.46 1.46 -1.88 -.09 -.09
round(sum(r$coefs * sim) - r$rho, 2)  # 1.04, class 1
```

Python

```python
r = SVC(kernel="rbf", gamma=0.05, C=1e5).fit(X, y)
print(r.support_)        # [6 10 11 0 1]
sim = np.exp(-0.05 * ((X[r.support_] - [4, 4]) ** 2).sum(1))
print(sim.round(3))      # .670 .427 .427 .905 .905
w = r.dual_coef_[0]      # signed weight of each one
print((w * sim).round(2))  # -1.88 -.09 -.09 1.46 1.46
print(round((w * sim).sum() + r.intercept_[0], 2))  # 1.04
```

Listing 4. R and Python: the vote of the support vectors for $(4, 4)$

γ is the reach of one case For the cases $(5, 3)$ and $(2, 2)$ the squared distance is $3^2 + 1^2 = 10$. With $\gamma = 0.05$ their similarity is $e^{-0.5} \approx 0.61$, and with $\gamma = 0.5$ it is $e^{-5} \approx 0.007$. Large γ makes each support vector influence only a small island around itself, which overfits the way $K = 1$ does in kNN. Refit the

running cases with $\gamma = 0.5$ and the model needs ten support vectors instead of five, and $(2, 2)$, eight squared units from $(4, 4)$, now counts with similarity $e^{-4} \approx 0.018$, almost nothing. Small γ makes every case look alike and the boundary nearly straight. γ and C both buy flexibility, so a large C can stand in for a large γ and the reverse. Tune them together (Section 7).

Boundaries with different kernels Figure 11 fits a linear and a radial SVM to two measurements of `iris` [5], after the scikit-learn gallery example [8]. The curved boundary comes from the kernel, not from a different algorithm. Neither separates versicolor from virginica, which overlap in these two measurements.

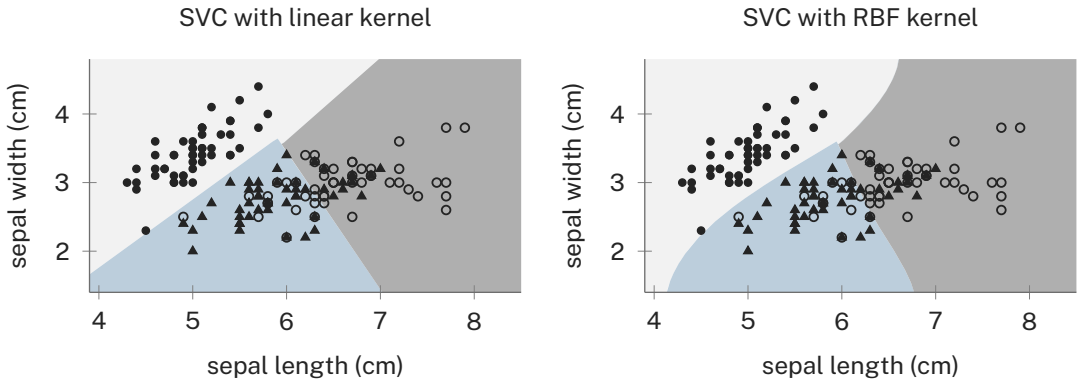


Figure 11. Decision regions on `iris` (sepal length and width, $C = 1$), with the linear kernel (left) and the radial kernel (right). After the scikit-learn gallery example [8].

6 Scaling is part of the model

The rule No SVM ignores units. The street is measured in the units of the predictors, and the radial kernel depends on squared distances, which the column with the widest numeric range dominates. Standardize every predictor, and fit the scaler on the training rows only, as for kNN. `e1071::svm` standardizes numeric columns for you (`scale = TRUE`). `sklearn.svm.SVC` does not, so put a `StandardScaler` in front of it in a pipeline.

Worked example Record x_1 of the running data in a unit ten times smaller, so every value is multiplied by 10. Nothing about the cases has changed, but the support vectors become rows 2, 7 and 11 instead of 1, 2 and 7, and the boundary moves.

R

```r
d10 <- transform(d, x1 = 10 * x1)  # new unit
s10 <- svm(y ~ x1 + x2, d10, kernel = "linear",
           cost = 1e5, scale = FALSE)
s10$index      # 2 7 11, no longer 1 2 7
```

Python

```python
X10 = X * [10, 1]     # x1 in a new unit
s10 = SVC(kernel="linear", C=1e5).fit(X10, y)
print(np.sort(s10.support_))  # [ 1  6 10], not [0 1 6]
```

Listing 5. R and Python: a change of unit changes the support vectors

Categorical predictors An SVM needs numbers. A categorical predictor enters as indicator columns, one per level. A predictor with dozens of levels becomes dozens of distance dimensions, which is one reason trees are often preferred when most predictors are categories.

7 Tuning C and γ inside, scoring outside

What it is for The cost C and the reach γ decide the model, the defaults are rarely right, and choosing them is fitting. Recall that the best score of a search is partly luck, so the score of the chosen setting must come from rows the search never saw. The protocol is nesting.

1. Cut the rows into five stratified **outer** folds.
2. Inside each outer training set, cut five stratified **inner** folds, and choose C and γ by their mean inner score.
3. Refit the chosen setting on the whole outer training set and score it once on its outer test fold.

The five outer scores estimate how well the whole procedure, search included, does on new rows.

The data The `Glass` data [6] has 214 glass fragments described by their refractive index and eight oxide contents and classified into six types, from 76 fragments of type 2 to 9 of type 6. Because the types are unequal, every score is **balanced accuracy**, the mean of the per-type recalls, so the small types count as much as the large ones. A model that always answers the most common type scores $1/6 = 0.167$.

Six types from a two-class method An SVM draws one street between two classes. With six glass types, `e1071::svm` and `SVC` both fit one SVM for every pair of types, $6 \times 5/2 = 15$ machines, each trained only on the fragments of its two types. A new fragment is scored by all 15, each votes for one of its two types, and the type with the most votes wins. The settings C and γ are shared by the 15 machines, so tuning them is still one search over one grid.

Fixing the folds Each language draws stratified folds once, with a seed, and reuses them for every model it compares, which is the pairing rule. Python's `StratifiedKFold` with `random_state=3141` gives the same split on every run, and the same object, applied to an outer training set, gives that set's inner folds. R's `createFolds` after `set.seed(3141)` does the same in R. The two languages draw different random numbers, so their folds, and therefore their fold scores, differ.

R

```r
data(Glass, package = "mlbench")
library(caret)
set.seed(3141)          # R's own folds
outer <- createFolds(Glass$Type, k = 5,
                     list = FALSE)
bacc <- function(y, p)    # balanced accuracy
  mean(tapply(p == y, y, mean))
grid <- expand.grid(gamma = 10^(-3:0),
                    cost = 10^(-1:4))
```

Python

```python
import pandas as pd
from sklearn.metrics import balanced_accuracy_score
from sklearn.model_selection import (
    GridSearchCV, StratifiedKFold)
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

url = ("https://archive.ics.uci.edu/ml/"
       "machine-learning-databases/glass/glass.data")
glass = pd.read_csv(url, header=None, index_col=0)
X = glass.iloc[:, :-1].to_numpy()
y = glass.iloc[:, -1].astype(str).to_numpy()
folds = StratifiedKFold(5, shuffle=True,
                        random_state=3141)
svm = make_pipeline(StandardScaler(), SVC())
grid = {"svc__C": 10.0 ** np.arange(-1, 5),
        "svc__gamma": 10.0 ** np.arange(-3, 1)}
```

Listing 6. R and Python: the data, the frozen folds and the grid

The grid C and γ act by multiplying, so the grid uses powers of ten. Table 6 shows the inner scores for outer fold 1. Along a diagonal, a larger C with a smaller γ scores about the same as a smaller C with a larger γ, because both buy the same flexibility. The top row is too stiff to separate the types,

and its 0.167 is that of a model answering one type for every case. The best cell, $C = 1000$ and $\gamma = 0.01$, sat on the edge of a first grid that stopped at $C = 1000$, so the grid was widened to $C = 10000$. A best setting on the edge of a grid is a reason to widen it.

Table 6. Inner cross-validated balanced accuracy over the C by γ grid, outer fold 1

$C \setminus \gamma$	0.001	0.01	0.1	1
0.1	0.167	0.167	0.278	0.176
1	0.167	0.376	0.575	0.479
10	0.377	0.532	0.629	0.523
100	0.495	0.642	0.649	0.519
1000	0.598	**0.742**	0.660	0.519
10000	0.707	0.725	0.660	0.519

R

```r
outer_score <- function(k) {
  d <- Glass[outer != k, ]
  inner <- createFolds(d$Type, 5, FALSE)
  cv <- function(g, C) mean(sapply(1:5,
    function(j) {
      m <- svm(Type ~ ., d[inner != j, ],
               gamma = g, cost = C)
      p <- predict(m, d[inner == j, ])
      bacc(d$Type[inner == j], p)
    }))          # choose INSIDE
  b <- grid[which.max(mapply(cv,
         grid$gamma, grid$cost)), ]
  m <- svm(Type ~ ., d, gamma = b$gamma, cost = b$cost)
  test <- Glass[outer == k, ]
  bacc(test$Type, predict(m, test))
}                # score OUTSIDE
scores <- sapply(1:5, outer_score)
round(scores, 3)
## 0.754 0.657 0.727 0.596 0.684
```

Python

```python
outer = []
for tr, te in folds.split(X, y):
    gs = GridSearchCV(svm, grid, cv=folds,    # INSIDE
        scoring="balanced_accuracy").fit(X[tr], y[tr])
    outer.append(balanced_accuracy_score(     # OUTSIDE
        y[te], gs.predict(X[te])))
print(np.round(outer, 3))
# [0.553 0.421 0.679 0.648 0.766]
```

Listing 7. R and Python: choose inside each outer training set, score on its outer fold

Python prints the outer scores 0.553, 0.421, 0.679, 0.648 and 0.766, mean 0.613. R, on its own folds, prints 0.754, 0.657, 0.727, 0.596 and 0.684, mean 0.684. Five folds of 214 fragments move the estimate by several points with the split alone, which is one more reason to compare models only on shared folds.

Differences to know

- `svm()` standardizes inside each fit. Python does it with the `StandardScaler` in the pipeline, which `GridSearchCV` refits on every training fold, so neither leaks test rows into the scaling.
- R's `scale` divides by the sample standard deviation and `StandardScaler` by the population one. On these folds the difference changes no choice.
- Passing the `StratifiedKFold` object as `cv` makes `GridSearchCV` use it for the inner folds. `GridSearchCV` breaks ties in favour of the first setting in its grid, so the R grid is listed in the same order.

8 Comparing with earlier models

Same folds for every model An SVM result means little without the methods it is meant to beat. Table 7 scores four models on the same five outer folds of the Python listing, each tuned inside. The baseline always answers the most common type. LDA has nothing to tune. kNN chooses K from 1 to 9, and the SVM chooses C and γ.

Table 7. Outer-fold balanced accuracy of four models on `Glass`

Outer fold	1	2	3	4	5
Baseline, most common class	0.167	0.167	0.167	0.167	0.167
LDA	0.574	0.613	0.452	0.608	0.481
kNN, K tuned	0.638	0.583	0.685	0.581	0.663
SVM, C and γ tuned	0.553	0.421	0.679	0.648	0.766
SVM minus kNN	−0.085	−0.162	−0.006	+0.067	+0.103

The means are 0.167, 0.546, 0.630 and 0.613. The SVM wins two folds and loses three to kNN.

Is the gap real? The corrected resampled t-test answers, with $1/4$ as the overlap term, because in five-fold cross-validation each fold tests one row for every four it trains on. The SVM minus kNN gives $t = -0.23$ and $p = 0.83$, so the gap is well within luck. Neither kNN nor the SVM can be told apart from LDA either on these folds ($p = 0.35$ and 0.63). All three beat the baseline by a wide margin.

R

```r
s <- data.frame(     # the Python fold scores
  lda = c(.574, .613, .452, .608, .481),
  knn = c(.638, .583, .685, .581, .663),
  svm = c(.553, .421, .679, .648, .766))
round(colMeans(s), 3)
## lda 0.546  knn 0.630  svm 0.613
corrected_t <- function(d, ratio) { # corrected
  ↪ resampled t-test
  v <- (1 / length(d) + ratio) * var(d)
  t <- mean(d) / sqrt(v)
  c(t = t, p = 2 * pt(-abs(t), length(d) - 1))
}
corrected_t(s$svm - s$knn, 1 / 4)  # 5 folds
##      t      p
## -0.228  0.831
```

Python

```python
from scipy import stats

s = pd.DataFrame({     # the fold scores above
    "lda": [.574, .613, .452, .608, .481],
    "knn": [.638, .583, .685, .581, .663],
    "svm": [.553, .421, .679, .648, .766]})
print(s.mean().round(3))
# lda 0.546  knn 0.630  svm 0.613
def corrected_t(d, ratio):  # corrected resampled t-test
    v = (1 / len(d) + ratio) * d.var(ddof=1)
    t = d.mean() / np.sqrt(v)
    return t, 2 * stats.t.sf(abs(t), len(d) - 1)
print(corrected_t(s.svm - s.knn, 1 / 4))  # 5 folds
# (-0.228, 0.831)
```

Listing 8. R and Python: the three means and the corrected test, on the fold scores of Table 7 typed in

The lesson is not that SVMs are weak. On 214 fragments, a more flexible model is not automatically a better one, and five folds of a small data set carry little evidence either way. Test the gap before you trust it.

9 Unequal classes

The cost C charges every class the same per unit of intrusion, so a type with nine fragments contributes little to the total fine, and the fit can afford to misclassify all of it. **Class weights** multiply the fine for each class. The usual choice, `class_weight="balanced"` in scikit-learn, gives every class the same total weight, and R's `class.weights` takes the same numbers as a named vector. Weighting is one more setting, not a fix, so choose it inside like C and γ, and judge it with the metric it is meant to improve. Balanced accuracy and per-class recall show the gain. Plain accuracy can hide it, or fall.

10 Common misconceptions

- **"Scaling is a tidiness step."** It changes the model, for every kernel including the linear one. R scales by default and Python does not, so the same analysis gives different answers in the two languages unless a scaler is added.
- **"The defaults are a reasonable model."** $C = 1$ and the default γ are a starting point, and on `Glass` the search moved C to 1000 or more. Tune inside cross-validation.
- **"Larger C is a stronger model."** Larger C fits the training rows more closely, and training error always rewards it. Choose C by cross-validation.
- **"The SVM is the most flexible model, so it wins."** On `Glass` it ties kNN and LDA within the corrected test.
- **"The decision value is a probability."** It is a score. It ranks cases, and it is not on a probability scale.

11 Summary

- An SVM fits the boundary in the middle of the widest street. Only the support vectors, the cases on or inside the street, decide it.
- The soft margin lets cases intrude for a fine of C per unit. Small C gives a wide, stiff street and large C a narrow, flexible one.
- A kernel is a similarity between two cases. The radial kernel is the default, and its γ sets how far one case reaches.
- Scale every predictor, with the scaler fitted on training rows only.
- Choose C and γ inside, score outside, and compare with earlier models on the same folds using the corrected test.
- Treat class weights as a setting when the classes are unequal, and report a class-aware metric.

References

[1] B. Boehmke and B. Greenwell. *Hands-On Machine Learning with R*. Chapter on support vector machines. Chapman and Hall/CRC, 2019.

[2] B. E. Boser, I. M. Guyon, and V. N. Vapnik. "A training algorithm for optimal margin classifiers". In: *Proceedings of the Fifth Annual Workshop on Computational Learning Theory (COLT)*. ACM, 1992, pp. 144–152.

[3] C.-C. Chang and C.-J. Lin. "LIBSVM: A Library for Support Vector Machines". In: *ACM Transactions on Intelligent Systems and Technology* 2.3 (2011), 27:1–27:27. The solver underneath both `e1071` and `sklearn.svm`.

[4] C. Cortes and V. Vapnik. "Support-vector networks". In: *Machine Learning* 20.3 (1995), pp. 273–297.

[5] R. A. Fisher. "The use of multiple measurements in taxonomic problems". In: *Annals of Eugenics* 7.2 (1936), pp. 179–188.

[6] B. German. *Glass Identification*. UCI Machine Learning Repository. Dataset. 1987.

[7] D. Meyer, E. Dimitriadou, K. Hornik, A. Weingessel, and F. Leisch. *e1071: Misc Functions of the Department of Statistics, Probability Theory Group (Formerly: E1071), TU Wien*. Version 1.7-17. Documentation for `svm` and `predict.svm`. CRAN. 2025.

[8] scikit-learn developers. *Plot Different SVM Classifiers in the Iris Dataset*. Gallery example. URL: `https://scikit-learn.org/stable/auto_examples/svm/plot_iris_svc.html`.

[9] scikit-learn developers. *Support Vector Machines*. User guide, with the `SVC`, `SVR` and `LinearSVC` API documentation. URL: `https://scikit-learn.org/stable/modules/svm.html`.

[10] V. N. Vapnik. *The Nature of Statistical Learning Theory*. Springer, 1995.